



UNIVERSIDAD DE
GUADALAJARA



Universidad de Guadalajara
Sistema de Educación Superior
Centro Universitario de Ciencias Exactas e Ingenierías

Arquitectura de Computadoras
Sección D05
Profesor: Jorge Ernesto Lopez Arce Delgado

A02 compuertas logicas

Ingeniería en Computación

Ortiz Segura Emmanuel Alejandro

221501336

31 de Enero de 2025

Introducción:

Emmanuel Alejandro Ortiz S.

26 01 26

Scribe

FPGA y verilog

FPGA: Field Programmable Gate Array

Investigar Arquitectura FPGA)

Cache: memoria statica → es mas cara, usa 6-7 transistores y no necesita refresh

Dram: mas barata, solo un transistor con un capacitor y es un bit.
Se necesitan 64 de esas para una palabra.

Verilog: creado en 1983 - Gateway

En 1989 Cadence adquiere verilog

En 1990 Dominio Público

En 1995 verilog se hace estándar IEE 1364

2001 → versión revisada IEE 1364

HDL: lenguaje descripción de hardware

Niveles de abstracción:

- Nivel Comportamental
- Nivel de Transferencia de Registros
- Nivel de Componentes lógicos

→ tipo de dato

Registro
net,
Parámetro

Puertas,
senales,
modulos

2 → alta impedancia, x valor desconocido

1 → verdadero 0 → falso

Arquitectura de una FPGA: Una FPGA se conforma principalmente de 3 partes.

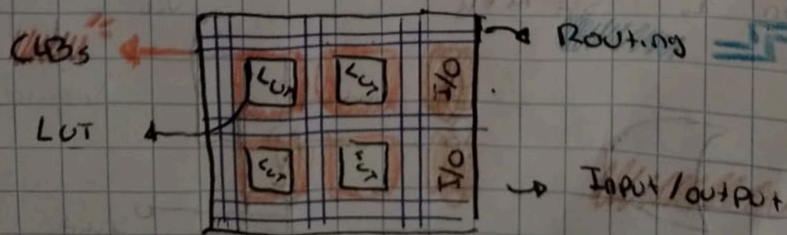
1. **CLBs (Configurable Logic Blocks)**: Aquí no hay compuertas lógicas físicas fijas. Hay **LUTs** (Look-Up Tables)

• LUT → Una pequeña memoria que almacena una tabla de verdad. Es decir que por ejemplo una compuerta AND NO SE "CONSTRUYE", simplemente se escribe la Tabla en el LUT

2. **Interconexiones Programables (Routing)**: Son como autopistas con interruptores. Permiten conectar la salida de un CLB con la entrada de otro.

3. **Bloques I/O**: Pines físicos que conectan el chip con el mundo exterior (Leds, botones ...)

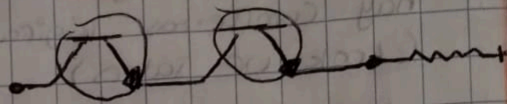
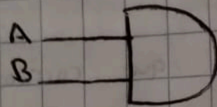
Verilog: es un HDL (Hardware Description Language). Cuando se escribe en verilog, la herramienta de síntesis (el software de la FPGA) lee la descripción y decide cómo conectar LUTs y los cables para crear ese circuito físico.



Compuertas lógicas:

AND: Multiplicación Booleana, es 1 cuando A y B son 1 lógico. $S = A \cdot B$ $\alpha \leftarrow \text{verilog}$

A	B	
0	0	0
1	0	0
0	1	0
1	1	1

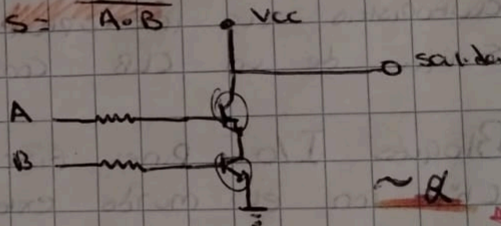
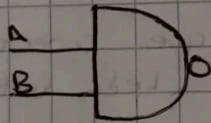


+ Son 2 en serie, fluye cuando ambos reciben 1

Tabla de verdad | símbolo | Arreglo de Transistores

NAND: Negación AND, es 1 cuando A y B tiene al menos un 0 lógico. $S = \overline{A \cdot B}$

A	B	
0	0	1
1	0	1
0	1	1
1	1	0

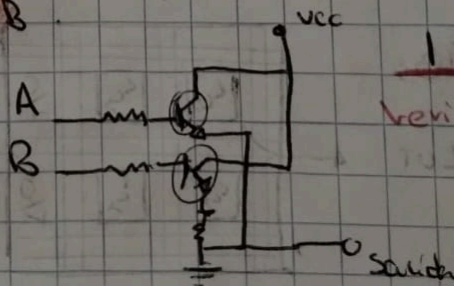
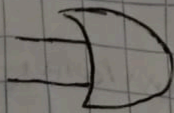


$\sim \alpha$
verilog

Tabla de verdad | símbolo | Arreglo de Transistores

OR: Suma Booleana, es uno cuando A o B o ambas son 1. $S = A + B$

A	B	
0	0	0
1	0	1
0	1	1
1	1	1

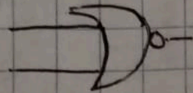


1
verilog

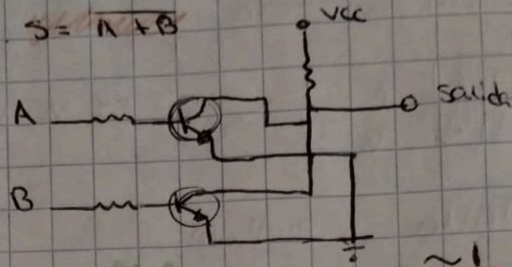
NOR: Es la negación de OR, Para que sea 1 Ambos deben ser 0. $S = \overline{A+B}$

A	B	
0	0	1
0	1	0
1	0	0
1	1	0

Tabla



Símbolo



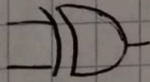
Arreglo

~ 1
verilog

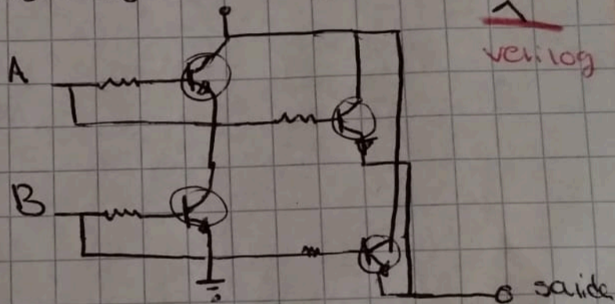
XOR: Or exclusiva, es 1 cuando lo tiene A ó B Pero No AMBOS. $S = A\bar{B} + \bar{A}B = A \oplus B$

0	0	0
0	1	1
1	0	1
1	1	0

Tabla



Símbolo



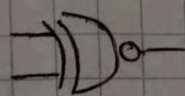
Arreglo

$\wedge$
verilog

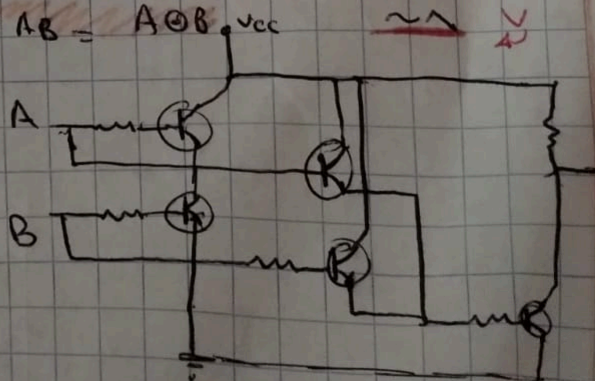
XNOR: Nor exclusiva es 1 cuando Ambos son iguales $S = \overline{A\bar{B}} + \overline{\bar{A}B} = A \odot B$

0	0	1
0	1	0
1	0	0
1	1	1

Tabla



Símbolo

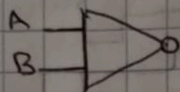


Arreglo

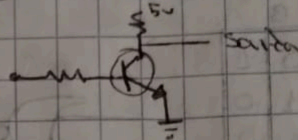
$\sim \wedge$
verilog

NOT: Es la negación booleana, cualquier entrada sera la negación o la contraria.

A
0
1



~ Verilog



Tabla

Simbolo

Arreglo de Transistores

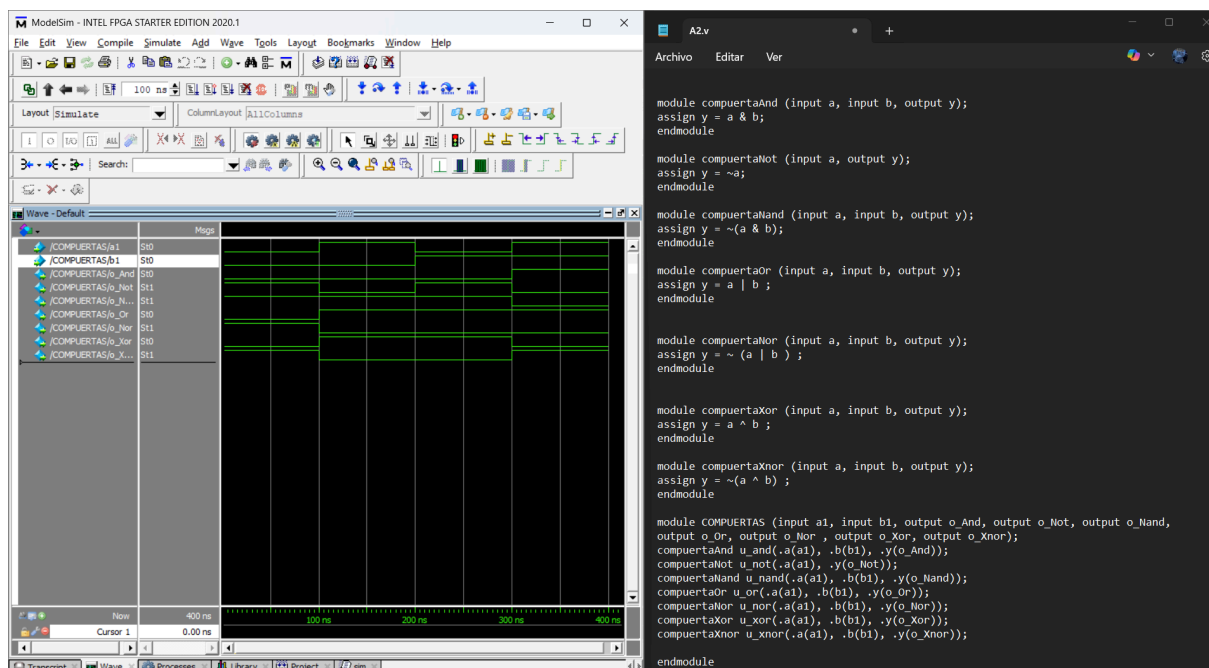
OBJETIVOS

Esta actividad es para que se tenga la oportunidad de corroborar la descarga e instalación correcta del compilador y simulador de HDL a utilizar durante el semestre, cualquier duda o error con el SW debe ser resuelto en durante esta práctica.

Así como comprender e investigar la sintaxis de cada uno de los operadores de lógica combinacional en el HDL elegido (Verilog o VHDL).

Las operaciones lógicas a implementar son las 7 compuertas lógicas principales AND, NAND, OR, NOR, NOT, XOR, XNOR, siendo la implementación a nivel de 1 bit, es decir operaciones lógicas de dos bits de entrada y uno de salida en cada uno (excepto la compuerta NOT)
Estas deben ser implementadas todas en un solo

Simulación y Código de compuertas en modelsim



Repositorio GitHub:

https://github.com/ItsEmmaNames/ArquitecturaComputa_doras.git

Conclusiones:

Las compuertas lógicas son esenciales en el diseño de cualquier circuito, en una FPGA es posible diseñar tus propios circuitos internos gracias a sus CLBs o “Configurable Logic Blocks” donde hay LUTs “Look up tables” que son pequeñas memorias que almacenan tablas de verdad, y es aquí donde se relaciona con las compuertas lógicas

El lenguaje que se utiliza es un lenguaje de descripción de hardware en este caso verilog HDL, donde su principal estructura son los módulos, la creación de ellos es sencilla ya que solo es prestar atención a la sintaxis, lo complicado de la actividad a mi parecer fue instanciar los distintos módulos a uno que implementa los otros, como se puede observar en la captura de pantalla del notepad.

Para ello, la documentación, algunos videos de youtube e incluso herramientas como la inteligencia artificial fueron clave para aprender que hay varias maneras de instanciar un módulo pero la más sencilla es llamar al módulo, añadirle un “apodo” y entre paréntesis poner .(puerto) y otros paréntesis (la señal) también evitar apodar con palabras reservadas como and, or, porque eso da errores.

Por último para un mejor diseño es bueno dibujar o representar cada módulo con sus entradas y salidas, esto permite que visualmente te des una idea de lo que estás haciendo como en este caso las compuertas que entendí que hay 2 inputs pero cada compuerta tiene su propia salida y eso “facilita” el código

Referencias bibliográficas

a1109. (2025, March 16). Verilog - Módulos y Simulación [Video]. YouTube.
<https://www.youtube.com/watch?v=aVlb0MFPhkg>

Visual Electric. (2021, March 31). The best way to start learning Verilog [Video]. YouTube.
https://www.youtube.com/watch?v=2lReMT_zjK8

AlienTux. (2025, August 4). ¿Cómo hacer módulos dentro de módulos en Verilog? [Video]. YouTube. <https://www.youtube.com/watch?v=xGFL0ABOMuQ>

Ruiz De Clavijo Vázquez, P., Departamento de Tecnología Electrónica, & Universidad de Sevilla. (2012). INTRODUCCIÓN a HDL VERILOG.
<https://www.dte.us.es/Members/paulino/Verilog-Intro.pdf>

Wtdtantakatan. (2016, September 21). Transistores en compuertas lógicas. Tantakatanblog.
<https://tantakatanblog.wordpress.com/2016/09/18/transistores/>

Compuertas lógicas – Sistemas Digitales. (n.d.).
<https://virtual.cuautitlan.unam.mx/intar/sistdig/compuertas-logicas/>

Schneider, J., & Smalley, I. (2025, November 27). Matriz de puertas programable en campo. IBM. <https://www.ibm.com/mx-es/think/topics/field-programmable-gate-arrays>